



栈

有 n 个数依次进栈，可以随机出栈，求出栈的所有可能情况。

输入：

3

输出：

5

当然，这是一道卡特兰数的题，但是：

(1) 万一你不记得卡特兰数的公式了呢

(2) 万一你不知道这是卡特兰数呢

所以，背公式还不如自己推导呢！

卡特兰数

简单来说，就是我们已经知道了1—i-1个数出栈的可能性想，需要我们求出i个数出栈的可能性。

假设此时第k个数已经入栈了，那么还剩下(n-k)个数，那么剩下(n-k)个数的入出栈方案数很简单就是 $h[n-k]$ ，那么已经入栈的k个数的方案数呢？ $h[k]$ ？肯定不是， $h[k]$ 表示的是前k个数出入栈种树，但是这里已经入栈的k个元素，有没有必要先出栈，再让k+1这个元素进来，所以，明显有问题。

那我们就换一种思路，假设最后一个出栈的数是k，那么说明 $[1 \cdots k-1]$ 都在k入栈之前出栈了，然后k再入栈，接下来再是 $[k+1 \cdots n]$ 入栈，然后 $[k+1 \cdots n]$ 出栈。那么此时k都把前后两部分隔离成了独立的两部分，在k入栈之前，前面部分已经入栈并且出栈了。在k之后的部分，后面部分比k后入栈并且先入栈。所以，公式就出来了：

$$h[n] = h[0] * h[n-1] + h[1] * h[n-2] + \cdots + h[n-1] * h[0]$$

卡特兰数

还有一种证明方法，也很简单：

设 $h[i][j]$ 表示栈内有 i 个，出栈的有 j 个的方案数。要达到当前这种状态，他的前一状态只有两种可能，入栈一个或者出栈一个，如果是入栈要到达 $h[i][j]$ ，那么上一状态就是 $h[i-1][j]$ ，如果是出栈到达 $h[i][j]$ ，那么就是 $h[i+1][j-1]$ 。那么递推公式就有了：

$$h[i][j] = h[i-1][j] + h[i+1][j-1].$$

注意一下，只有 j 是递增的，可能后用到后面的值，所以先枚举 j ，在枚举 i 。就可以得到正确答案。

但是前面两种时间复杂度都是 $O(n^2)$ 的，能不能更快一点，直接找公式而不是递推公式。

卡特兰数

公式也是可以推导的。画一个图就很好理解了。

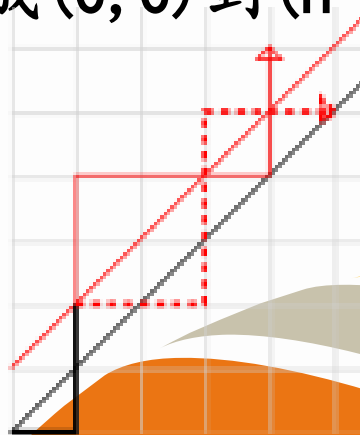
在平面直角坐标系中，x轴表示入栈次数，y轴表示出栈次数，那么卡特兰数就可以表示为从(0,0)点到达(n,n)点不越过直线 $y=x$ 的方案数。

如果不考虑限制，总方案数为 $C(2n, n)$ ，那么限制呢？我们会发现对于所有不满足条件的情况，一定是在若干次操作之出栈比入栈多一次，即这个点在 $y=x+1$ 上，我们发现如果把后面部分都相对于这条线对称，那么最后一定要到达点 $(n-1, n+1)$ 。即任何非法路径都可以变成(0,0)到 $(n-1, n+1)$ 的一条无限制的情况，那么合法方案为：

$$C(2n, n) - C(2n, n-1) :$$

$$\text{也可以转换成 } 1/(n+1) * C(2n, n)$$

$$\text{还有 } h[i+1] = (4n+2)/(n+2) * h[i]$$



卡特兰数

也可以这样考虑，对于01序列，如果不满足0的数量永远大于1的数量，那么肯定是从某一个1开始之后的数，后面有 k 个1， $k+1$ 个0。

比如011001

从第2个1开之后0的数量比1的数量多一个

那么这种情况有多少种呢？我们发现如果我们把后面的0都变成1，1都变成0，那么最终答案有 $k+1$ 个1， $k-1$ 个0. 这两种方案是一一对应的。

也就是对于一个长度为 $2*n$ 的序列，其中有 $n+1$ 个1， $n-1$ 个0，他都可以转换成唯一对应的不满足卡特兰数的序列。

比如110011，找到第一个1的数量 >0 的数量的位置，再把后面的数取反，得到的序列就是非卡特兰数的序列

那么实际上长度为 $2*n$ 的序列中，不满足条件的情况就是 $C(2n, n-1)$ 种，所以总的符合条件的数有 $C(2n, n) - C(2n, n-1)$

卡特兰数

根据前面的推导过程，我们很容易想到拉特兰数的适用范围：

(1) 一种情况的数量一直 $\geq$ 另一种情况

(2) 最终两种情况的数量一定相等

只要满足这两种条件，那么他就是卡特兰数

(1) 给出一个 n ，要求一个长度为 $2n$ 的01序列，使得序列的任意前缀中的1的个数不少于0的个数

(2) 从 $(0, 0)$ 到达 (n, n) 点，每次只能向上或者向右，问不越过 $y=x$ 的方案数

(3) 有 $2n$ 个人进入剧场，入场费5元，售货员自己没零钱有 n 个人有5元的钞票，剩下 n 个人只有10元钞票，问有多少种方法可以使得所有人都可以正常入场。

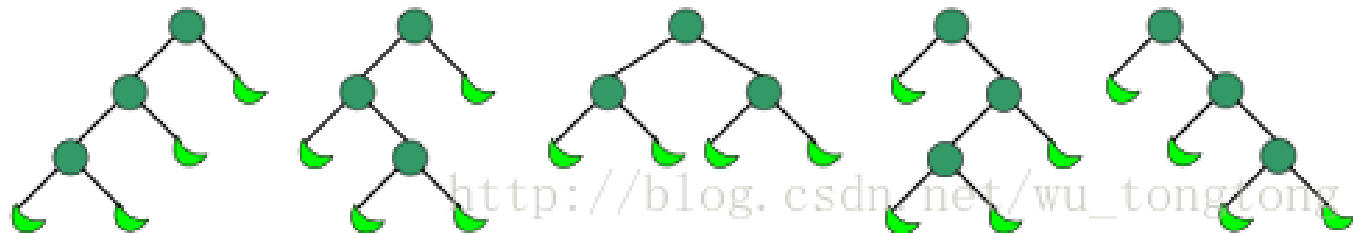
(4) n 个左括号， n 个右括号，问合法括号排列的方案数

卡特兰数

一共有 $2n+1$ 个结点，问所有不同构的含有 $(n+1)$ 个叶子结点的二叉树的个数(区分左右儿子)。

对于一个满二叉树而言，叶子结点个数=非叶子结点数+1
但是实际上，也有不是满二叉树就满足条件。

只要满足子树要么为空，要么满都可以满足条件。设叶子结点为 x ，非叶子结点为 y ；如果子树都满或者空， $x=y+1$ ，如果不满足， $x < y+1$ ；并且只有当插入到只有一个儿子结点的位置，叶子结点才会+1。所以一定是满或空这种形态。



卡特兰数

那么我们思考一下，最开始只有一个结点，就是叶子结点，然后再添加一个结点，此时 $x=y$ ，那么为了满足 $x=y+1$ ，又的让叶子结点+1。一直这样循环。当最后满足有 n 个叶子结点， n 个非叶子结点的时候，最后再添加的那一个结点是固定的，所以实际上还是卡特兰数。只是：

$h[i]$ 表示有 $n+1$ 个叶子结点的二叉树的个数。

卡特兰数

把一个凸 n 边形划分成 $(n-2)$ 个三角形的方案数：

卡特兰数，先固定一条边，再从剩下点里面选择一个点，那么这样构成的一个三角形会把整个凸 n 边形分成两个独立的多边形，那么这里两个多边形的划分方案乘起来就是当前这种情况的总方案，那么由于选择的点只要不和这条边的两个点重合，那么都可以。实际上就是 $h[i+2]$ 表示卡特兰数的第 n 项的答案。

练习题

洛谷 1044 栈
洛谷 1976 鸡蛋饼
洛谷 2532 树屋阶梯
洛谷 3200 有趣的数列
洛谷 4981 父子
BZOJ 1856
BZOJ 4359
POJ 1095
POJ 2084
HDU 1267 下沙的沙子有几粒
HDU 2067 小兔的棋盘
HDU 1133 buy the Ticket

矩阵

矩阵就是一个数字列阵，一个n行r列的矩阵可以表示为：

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

矩阵的简单运算

矩阵加法：

矩阵加法很简单，就是把两个矩阵每个相同位置上的相加，第一个矩阵为A，第二个矩阵为B，那么就是 $A_{ij}+B_{ij}$ 。

很明显，两个矩阵做加减法必须要满足的条件是这两个矩阵的行列相同，否则就不可以进行加减运算，最终的得到的矩阵的行列和原矩阵相同。

$$\begin{bmatrix} 1 & 3 \\ 1 & 0 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 7 & 5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1+0 & 3+0 \\ 1+7 & 0+5 \\ 1+2 & 2+1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 8 & 5 \\ 3 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 3 \\ 1 & 0 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 7 & 5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1-0 & 3-0 \\ 1-7 & 0-5 \\ 1-2 & 2-1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ -6 & -5 \\ -1 & 1 \end{bmatrix}$$

矩阵的简单运算

矩阵乘法：

矩阵乘法稍微复杂一点，比如 $A*B$ ，相当于A矩阵的第i行的每个数与B矩阵的第j列对应位置的每个数相乘当做新矩阵[i, j]位置的值。

$$A \times B = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \times B = \begin{pmatrix} a_1 \times B \\ a_2 \times B \\ a_3 \times B \end{pmatrix} = \begin{pmatrix} a_{11} \times b_1 + a_{12} \times b_2 + a_{13} \times b_3 \\ a_{21} \times b_1 + a_{22} \times b_2 + a_{23} \times b_3 \\ a_{31} \times b_1 + a_{32} \times b_2 + a_{33} \times b_3 \end{pmatrix}$$

$$\begin{bmatrix} 2 & 4 & 5 \\ 1 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 3 & 4 \\ 1 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 2 \times 3 + 1 \times 4 + 2 \times 5 & 2 \times 4 + 3 \times 4 + 5 \times 1 \\ 1 \times 3 + 3 \times 1 + 4 \times 2 & 1 \times 4 + 3 \times 3 + 4 \times 1 \end{bmatrix} = \begin{bmatrix} 20 & 25 \\ 14 & 17 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 4 \\ 1 & 3 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 4 & 5 \\ 1 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 \times 2 + 1 \times 4 & 3 \times 4 + 1 \times 3 & 3 \times 5 + 1 \times 4 \\ 1 \times 2 + 3 \times 1 & 1 \times 4 + 3 \times 3 & 1 \times 5 + 3 \times 4 \\ 2 \times 2 + 1 \times 1 & 2 \times 4 + 1 \times 3 & 2 \times 5 + 1 \times 4 \end{bmatrix} = \begin{bmatrix} 10 & 15 & 19 \\ 5 & 13 & 17 \\ 5 & 11 & 14 \end{bmatrix}$$

矩阵的简单运算

所以矩阵的乘法首先必须要满足的条件是：

如果A是一个n行r列的矩阵，B是一个r行m列的矩阵，用A矩阵每一行去乘以B矩阵的每一列，那么最终会得到一个n行m列的矩阵。

矩阵乘法不满足交换律，首先是r行m列的矩阵无法和一个n行r列的矩阵相乘，因为不满足条件；其次，一个n行m列的矩阵*一个m行n列的矩阵得到一个n行n列的矩阵；一个m行n列的矩阵*n行m列的矩阵得到一个m行m列的矩阵，完全没有任何联系。

但是矩阵的乘法满足结合律，即 $(A*B)*C=A*(B*C)$

单位矩阵

那么思考一下，一个方阵A(n行n列)要乘以一个什么矩阵才能等于他本身？

$$\begin{bmatrix} 2 & 3 \\ 1 & 5 \end{bmatrix} \times \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 5 \end{bmatrix}$$

① $2a+3c=2$
② $3b+3d=3$
③ $a+5c=1$
④ $b+5d=5$

①③联立: $\begin{cases} a=1 \\ c=0 \end{cases}$
②④联立: $\begin{cases} b=0 \\ d=1 \end{cases}$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

实际上对于任意的一个矩阵，他乘以只有主对角线为1的矩阵，都等于它本身。只有主对角线为1，其他都为0的矩阵，称之为单位矩阵。

方阵乘幂

方阵乘幂是指， A 是一个方阵，将 A 连乘 N 次，即 $C=A^n$ 。

注意：只有 A 是方阵的时候才可以乘幂。

那么我们应该如何快速进行乘幂计算呢？

因为矩阵乘法满足结合律，那么我们完全可以考虑快速幂的做法来进行方阵快速乘幂。

```
node js(node t1,node t2)
{
    node t3;
    for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
    t3.map[i][j]=0; //一定要初始化
    for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
    for(int k=1;k<=n;k++)
    t3.map[i][j]=(t3.map[i][j]+t1.map[i][k]*t2.map[k][j])%p;
    return t3;
}
```

方阵乘幂

只要矩阵乘法会了，矩阵快速幂就简单了。参照整数的快速幂。

```
node quick(int x)
{
    if(x==0)
        return e; // 返回一个单位矩阵
    node ans=quick(x/2);
    ans=js(ans,ans); // 两个矩阵相乘
    if(x&1)
        ans=js(ans,a);
    return ans;
}
```

斐波那契数列

看上去矩阵快速幂有点鸡肋，在生活中并没有太大的实际意思，但是如果我们可以把某种状态转移成一个矩阵，再把他的递推式转移成矩阵的乘幂，那么我们就可以log的时间复杂度计算出一个递推式的答案。

比如：求解斐波那契数列的第n项， $n \leq \text{long long}$ 。

很明显，线性是无法求解的，那么是不是就没有更快的算法了呢？并不是。我们可以尝试是否可以把当前状态转移成一个矩阵，把递推公式转移成矩阵乘幂。转移成状态很简单，因为我们只有知道前两个状态才能知道后一个状态，那么矩阵可能就是

$$\begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix}$$

$$\begin{bmatrix} F[i+1] \\ F[i] \end{bmatrix}$$

斐波那契数列

左边是一个2行1列的矩阵，右边也是一个2行1列的矩阵，如果要成一个数字矩阵(因为涉及到幂，所以只能是方阵)，有两种可能，第一种是1行1列，第二种是2行2列(在前面和在后面)，1行1列肯定没有办法满足条件。

$$\begin{bmatrix} F[i+1] \\ F[i] \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix}$$

$$a * F[i] + b * F[i-1] = F[i+1]$$

$$c * F[i] + d * F[i-1] = F[i], \text{ 那么结果就出来了。}$$

$$a=1, b=1, c=1, d=0;$$

$$\begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-2} * \begin{bmatrix} F[2] \\ F[1] \end{bmatrix}$$



斐波那契数列

求斐波那契数列前 n 项之和, $n \leq 2 \times 10^9$, 答案对 m 取余,
 $m \leq 10^9 + 10$;

输入: 5 1000

输出: 12



斐波那契数列

该题先一看，太简单了，直接用矩阵加速斐波那契数列，但是又一看，求和...对每一项进行矩阵快速幂求解？还不如暴力递推呢。所以我们要换一种思路。

比如找下斐波那契数列 $S[i]$ 和 $S[i-1]$ 的递推关系。

构造的矩阵里面肯定要有 $S[i]$, $F[i]$ ，但是只有这两个是肯定不够的，因为没有办法转换，那么肯定需要一个类似 $F[i-1]$ 这样的东西，才能把他转换成下一项。

$$\begin{bmatrix} S[i] \\ F[i] \\ F[i-1] \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} * \begin{bmatrix} S[i-1] \\ F[i-1] \\ F[i-2] \end{bmatrix}$$

斐波那契数列

很容易计算出第一行分别是(1, 1, 1), 第二行是(0, 1, 1), 第三行是(0, 1, 0), 我们在计算过程发现, 当第一行的值确定之后, 对后面是没有任何影响的, 相当于每行之间是相互独立的, 所以以后遇到类似问题就比较容易推导了。

当然, 很多时候推导的公式不止一种, 如果你的矩阵不是($S[i]$, $F[i]$, $f[i-1]$), 那么你的数字矩阵也会有一点变化。



佳佳的 Fibonacci

求 $T[i] = (F[1] + 2 * F[2] + \cdots + n * F[n]) \% m$ 的答案



佳佳的 Fibonacci

和上题一样，首先我们已知 $T[i]$ 和 $F[i]$ 的关系，那么就可以推导出 $T[i]$ 和 $T[i-1]$ 的关系， $T[i]=T[i-1]+iF[i]$ 但是很复杂， $iF[i]$ 中两个变量相乘，那我们看下是否可以推导出 $T[i]$ 和 $S[i]$ 的递推关系。

$$T[n]=F[1]+2F[2]+3F[3]+\cdots nF[n];$$

$$nS[n]=nF[1]+nF[2]+\cdots nF[n];$$

$$nS[n]-T[n]=(n-1)F[1]+(n-2)F[2]+\cdots F[n];$$

$$\text{我们设 } P[n]=nS[n]-T[n];$$

那么 $P[n]=P[n-1]+S[n-1]$ ，那么这个公式就和上一道题是一样的了。接下来就可以来推导矩阵了。

佳佳的 Fibonacci

矩阵很容易推导出来

1 1 0 0 0 1 1 1 0 0 1 1 0 0 1 0

以后遇到求解一个多项式的结果，如果他是一个递推式，那么一定可以转换成矩阵的形式。即：

$$F[n] = a_1 * F[n-1] + a_2 * F[n-2] + \dots + a_k * F[n-k].$$

这是一个k阶系数线性递推关系。

并且设计到的常数方阵也是k阶的。

但是，如果给的不是一个递推式呢？一般情况下，如果一个等式他是发散序列，那么他很难写成递推式的形式，如果他是线性或者收敛行的序列，那么他就可以构造递推式。对于发散序列，可以先把转换成线性或者收敛的序列。



练习题

洛谷 1962 斐波那契数列

L0J 10221 斐波那契前n项和

L0J 10222 佳佳的Fibonacci

洛谷 2233 公交车路线

洛谷 2886 牛继电器

BZOJ 1009 GT考试

BZOJ 1297 迷路

BZOJ 2973 石头游戏



高斯消元

高斯消元是一种求解线性方程组的方法。所谓线性方程组，是由M个N元一次方程共同构成的。线性方程组的所有系数可以写成一个M行N列的“系数矩阵”，再加上每个方程等号右侧的常数，可以写成一个M行N+1列的“增广矩阵”。

$$\begin{cases} x + 2y + z = 7 & A \\ 2x - y + 3z = 7 & B \\ 3x + y + 2z = 18 & C \end{cases}$$

转化为矩阵后为：

$$\left| \begin{array}{cccc} 1 & 2 & 1 & 7 \\ 2 & -1 & 3 & 7 \\ 3 & 1 & 2 & 18 \end{array} \right|$$

高斯消元求解

对于一个“增广矩阵”要如何求解呢？我们都知道，对于 n 元一次方程，当方程组数至少为 n 时才会有解（可能无解，可能有多组解）。

高斯消元法：

(1) 加减消元，比如如果要消去第一项，那么完全可以让两个 n 元一次等式的第一项系数互为相反数，那么，当这两项相加得到的新的方程就只有 $n-1$ 项了。

(2) 一直消下去，如果有解，那么一定有一个方程只有一个未知数，那么就可以得到答案，再回带求出其他解。

高斯消元过程

0.求解方程组

有这样一个三元一次方程组：

$$\begin{cases} 2x + y + z = 1 & \textcircled{1} \\ 6x + 2y + z = -1 & \textcircled{2} \\ -2x + 2y + z = 7 & \textcircled{3} \end{cases}$$

1.消去x

① $\times (-3)$ + ②得到

$$0x - y - 2z = -4$$

① + ③得到

$$0x + 3y + 2z = 8$$

高斯消元过程

从而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ 0x - y - 2z = -4 & \text{②} \\ 0x + 3y + 2z = 8 & \text{③} \end{cases}$$

2.消去y

② $\times$ 3 + ③得到

$$0x + 0y - 4z = -4$$

进而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ 0x - y - 2z = -4 & \text{②} \\ 0x + 0y - 4z = -4 & \text{③} \end{cases}$$

至此，我们已经求解出来了

$$z = 1$$

高斯消元过程

3.回代求解y

将 $z = 1$ 代入②, 求得

$$y = 2$$

进而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ y = 2 & \text{②} \\ z = 1 & \text{③} \end{cases}$$

最终得到

$$\begin{cases} x = -1 & \text{①} \\ y = 2 & \text{②} \\ z = 1 & \text{③} \end{cases}$$

至此, 整个方程组就求解完毕了。

矩阵模拟

三、再解算法

对于方程组，其系数是具体存在矩阵(数组)里的，下面在给出实际在矩阵中的表示（很熟悉就可以跳过不看啦~）

0.求解方程组

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 6 & 2 & 1 & -1 \\ -2 & 2 & 1 & 7 \end{bmatrix}$$

1.消去x

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 3 & 2 & 8 \end{bmatrix}$$

2.消去y

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -4 \end{bmatrix}$$

矩阵模拟

3.回代求解y

回代的时候，记录各个变量的结果将保存在另外一个数组当中，故保存矩阵的数组值不会发生改变，该矩阵主要进行消元过程。

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -4 \end{bmatrix}$$

四、再再解算法

说了这么多，其实有一些情况我们还没有说到。

通过上述的消元方法，其实我们比较希望得到的是一个上三角阵(省去了最后的val)

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & -1 & -2 \\ 0 & 0 & -4 \end{bmatrix}$$

下面问题来了：

解的情况

我们都知道，方程组不一定有解，那么当我们使用高斯消元的时候，怎么才能判断是否有解呢？

(1) 如果是一个完美的上三角，那么方程肯定是有唯一解集的

(2) 如果在消元过程中，存在有一行，各项系数都为0，但是常数不为零，那么很明显就是无解

(3) 如果有消元完成之后，有几项系数都为0，并且常数项也为0，那么这几项都可以随意取值，所以方程解不唯一。

(4) 并不是说 n 元一次方程有 n 个方程就一定有唯一解或者无解，也有可能有多组解

异或方程

高斯消元不但可以求解常规方程，还可以求解异或方程。

$$(a[1][1]*x[1]) \wedge (a[1][2]*x[2]) \wedge \dots \wedge (a[1][n]*x[n]) = y[1]$$

$$(a[2][1]*x[1]) \wedge (a[2][2]*x[2]) \wedge \dots \wedge (a[2][n]*x[n]) = y[2]$$

.....

$$(a[i][1]*x[1]) \wedge (a[i][2]*x[2]) \wedge \dots \wedge (a[i][n]*x[n]) = y[i]$$

.....

$$(a[m][1]*x[1]) \wedge (a[m][2]*x[2]) \wedge \dots \wedge (a[m][n]*x[n]) = y[m]$$

异或方程

1. 对于 $k=0..N-1$ ，找到一个 $M[i][k]$ 不为0的行 i ，把它与第 k 行交换。

2. 用第 k 行去异或所有 $M[i][j]$ 不为0的行 i ，消去它们的第 k 个系数，这样就将原矩阵化成了上三角矩阵；

3. 最后一行只有一个未知数，这个未知数就已经求出来了，用它跟上面所有含有这个未知数的方程异或，就消去了所有的这个未知数。

此时倒数第二行也只有一个未知数，它就被求出来了，用这样的方法可以自下而上求出所有未知数。

用矩阵表示大概就是这个样子

线性空间

在一个平面内，给定若干个向量 $a_1, a_2 \cdots a_n$ ，若向量 b 能由 $a_1, a_2 \cdots a_n$ 经过向量加法和标量乘法运算得到，则称向量 b 能被向量 $a_1, a_2 \cdots a_n$ 标出，这样 $a_1, a_2 \cdots a_n$ 能表出的所有向量构成一个线性空间。

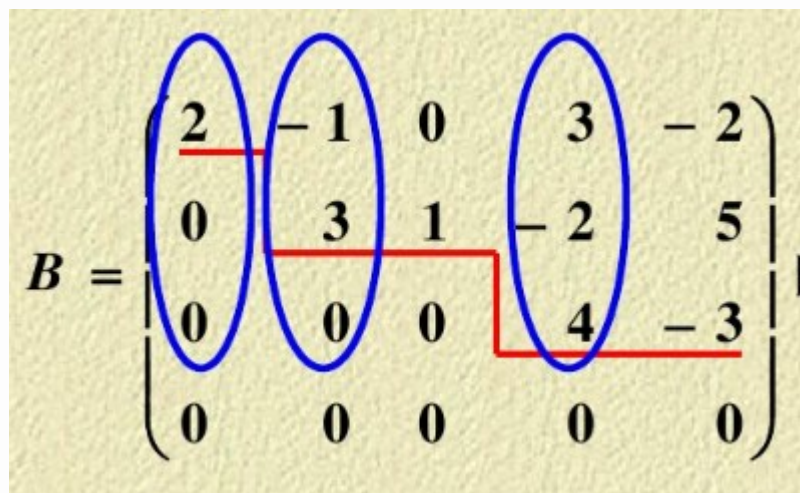
任意选出线性空间内的若干个向量，如果其中存在一个向量能被其他向量标出，则称这些向量线性相关，否则称为线性无关。

线性无关的极大子集被称为线性空间的基。简单来说基就是线性空间的极大线性无关子集。一个线性空间内的所有基包含的向量个数都相等，这个数被称为维数。

比如平面直角坐标系中的所有向量构成一个二维线性空间，他的一个基就是单位向量集合 $(0, 1), (1, 0)$ 。

矩阵的秩

关于矩阵的很多性质在大学线性代数中有详细解释，矩阵有一个很重要的东西叫矩阵的秩。秩这个东西的解释很复杂，只需要他的计算方法就可以了，就是把矩阵转换成上(下)三角形形式，不为0的行数就叫矩阵的秩。直观理解就是台阶数。下图该矩阵的秩就是3.



The matrix B is shown with its elements arranged in a 5x5 grid. The first three rows are non-zero, and the last two rows are all zeros. The non-zero rows are highlighted by blue ovals around the leading elements: 2, 0, and 0 in the first column. Red lines connect the leading elements of the non-zero rows to the right, forming a staircase pattern. The matrix is:

$$B = \begin{pmatrix} 2 & -1 & 0 & 3 & -2 \\ 0 & 3 & 1 & -2 & 5 \\ 0 & 0 & 0 & 4 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

装备购买

对于一个 n 行 m 列的矩阵，我们可以把它的每一行看做一个长度为 m 的向量，称为“行向量”，矩阵的 n 个行向量能够表出的所有向量构成一个线性空间。

脸哥最近在玩一个游戏，游戏里面有 n 件装备，每件装备有 m 个属性，用向量 $z[i]=(a_1, a_2 \cdots a_m)$ 表示，每件装备需要花费 c_i 。现在脸哥想用尽量少的钱买尽量多的装备。

如果一件装备的属性能够用购买的其他装备组合出(用手上已经购买的装备组合出这个效果，可以乘以一个常数)，那么这件装备就没有必要买了。

脸哥想要在买下最多数量的装备的前提下花最少的钱，问如何操作？

装备购买

该题如果明白了矩阵的秩，那么实际上就是求矩阵的秩，或者说求矩阵的非零行的个数。

关键怎么求最少要花多少钱呢？

我们想一下，当我们消到第 i 元的时候，此时下面有很多个前 $(i-1)$ 列都为0，第 i 列不为0的行，那么应该选择哪一个呢？采取贪心策略是正确的，就是用最小的一个价格消去其他的第 i 列的值，相当于选择一个价格最小的装备取尝试组成其他装备。

所以该题就解决了，虽然证明我们还不会。第一步，高斯消元，求出非零行的个数。在消元过程中，每次选择价格最便宜的装备当做基准去消元。

装备购买

该题如果明白了矩阵的秩，那么实际上就是求矩阵的秩，或者说求矩阵的非零行的个数。

关键怎么求最少要花多少钱呢？

我们想一下，当我们消到第 i 元的时候，此时下面有很多个前 $(i-1)$ 列都为0，第 i 列不为0的行，那么应该选择哪一个呢？采取贪心策略是正确的，就是用最小的一个价格消去其他的第 i 列的值，相当于选择一个价格最小的装备取尝试组成其他装备。

所以该题就解决了，虽然证明我们还不会。第一步，高斯消元，求出非零行的个数。在消元过程中，每次选择价格最便宜的装备当做基准去消元。

装备购买

输入：

3 3

1 2 3(每行表示属性)

3 4 5

2 3 4

1 1 2(每件装备价格)

输出：

2 2

1	2	3	1
0	-2	-4	1
0	-1	-2	2

1	2	3	1
0	-2	-4	1
0	0	0	2

最终矩阵的秩只有2，并且使用了第一行和第二行
(当前花费最小的装备)去消元，所以花费为1+1.



练习题

洛谷	3389	高斯消元法
洛谷	4035	球形空间产生器
BZOJ	1830	开关问题
POJ	2811	熄灯问题
洛谷	2962	Light
洛谷	3164	和谐矩阵
洛谷	4111	小Z的房间
洛谷	3265	装备购买
洛谷	4208	最小生成树计数
HDU	3949	XOR

